

Algorytmy Zaawansowane

Opracowane na podstawie wykładów Michała Tuczyńskiego

Spis treści

1	Wykład 2	2
1.1	Notacja asymptotyczna — przypomnienie	2
1.2	Strategia typu <i>Dziel i Zwyciężaj</i> (<i>Divide and Conquer</i>)	3
2	Wykład 3	6
2.1	Algorytm Strassera	6
2.2	Własność optymalnej podstruktury	7
2.3	Znajdywanie pary najbliższych punktów	7
3	Wykład 4 i 5	9
3.1	Programowanie dynamiczne	9
3.2	Algorytm mnożenia macierzy	10
3.3	Problem mnożenia ciągu macierzy	10
3.4	Problem najdłuższego wspólnego podciągu	15
4	Wykład 6	17
4.1	Algorytmy Zachłanne	17
5	Wykład 7	19
5.1	Kody Huffmana	19
5.2	Algorytm Huffmana	21
6	Wykład 9	23
6.1	Matroidy	23
6.2	Problem szeregowania zadań	25

Wykład 2

Notacja asymptotyczna — przypomnienie

$$f, g : \mathbb{N} \rightarrow \mathbb{R}_+$$

$$g(n) = \mathcal{O}(f(n)) \Leftrightarrow \exists_{c \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} g(n) \leq c \cdot f(n)$$

$$g(n) = \Omega(f(n)) \Leftrightarrow \exists_{c \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} c \cdot f(n) \leq g(n)$$

$$g(n) = \Theta(f(n)) \Leftrightarrow \exists_{c_1, c_2 \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} c_1 \cdot f(n) \leq g(n) \leq c_2 \cdot f(n)$$

$$g(n) = \mathcal{O}(f(n)) \Leftrightarrow f(n) = \Omega(g(n))$$

$$g(n) = \Theta(f(n)) \Leftrightarrow (g(n) = \mathcal{O}(f(n)) \wedge g(n) = \Omega(f(n)))$$

$$n^a = \mathcal{O}(n^b), \quad 0 < a \leq b$$

$$\log n = \mathcal{O}(n^\epsilon), \quad \epsilon > 0$$

$$n^a = \mathcal{O}(b^n), \quad 1 < a \leq b$$

$$\log_a n = \Theta(\log_b n), \quad a, b > 1$$

$$n^a = \mathcal{O}(b^n), \quad 0 < a, \quad b > 1$$

$$T(n) = 5n^3 + 2n^2 + 100n \log n \underset{n \rightarrow \infty}{\approx} 5n^3 = \Theta(n^3)$$

Procedura rekurencyjna

$$n! = \begin{cases} 1 & n = 0 \\ n \cdot (n-1)! & n \geq 1 \end{cases}$$

$$\text{Oczywiste jest, że } n! = \prod_{i=1}^n i$$

Ciąg Fibonacciego F_n - ciąg Fibonacciego.

$$F_n = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F_{n-1} + F_{n-2} & n \geq 2 \end{cases}$$

Algorytm 1: Rekurencyjna konstrukcja ciągu Fibonacciego

```

function FIBB( $n$ )
1:   if  $n = 0$  then return 0
2:   if  $n = 1$  then return 1
3:   return FIBB( $n - 1$ ) + FIBB( $n - 2$ )

```

Wywołanie rekurencyjne dla (typowo łatwiejszych i mniejszych) instancji tego samego problemu.

Strategia typu *Dziel i Zwyciężaj* (*Divide and Conquer*)

Algorytm typu *dziel i zwyciężaj* składa się z trzech części

1. *Dziel* – problem jest dzielony na mniejsze podproblemy
2. *Zwyciężaj* – podproblemy są rozwiązywane rekurencyjnie
3. *Połącz* – rozwiązanie całego problemu jest konstruowane z rozwiązań podproblemów

Zakładamy, że (zazwyczaj) w fazie *dziel* problem jest dzielony na co najmniej dwa podproblemy i podproblemy są rozłączne.

Przykład 1.1. Mnożenie liczb n -cyfrowych x, y – liczby naturalne n -cyfrowe.

Aby policzyć $x \cdot y$ można użyć mnożenia pisemnego (jak na kartce). Tym sposobem musimy wykonać $\mathcal{O}(n^2)$ mnożeń cyfr. Spróbujemy znaleźć lepszą metodę. Dzielimy x i y na „dwie połowy”. Zakładamy, dla uproszczenia, że n jest parzyste.

$$x = x_1 x_2 \cdots x_n = \underbrace{x_1 x_2 \cdots x_{n/2}}_{x_L} \underbrace{x_{n/2+1} \cdots x_n}_{x_R} = 10^{n/2} x_L + x_R$$

podobnie dla y :

$$y = \cdots = 10^{n/2} y_L + y_R$$

Wymnażamy:

$$x \cdot y = (10^{n/2} x_L + x_R) \cdot (10^{n/2} y_L + y_R) = 10^n x_L y_L + 10^{n/2} (x_L y_R + x_R y_L) + x_R y_R$$

Złożoność czasowa algorytmu – ozn. $T(n)$.

- Podział x, y na x_L, x_R, y_L, y_R zajmie $\mathcal{O}(n)$ czasu.
- Dodanie liczb n -cyfrowych zajmie $\mathcal{O}(n)$ czasu.
- Mnożenie przez 10^n jest równoważne dopisaniu n zer więc też $\mathcal{O}(n)$.

$$T(n) = 4 \cdot T(n/2) + \Theta(n)$$

Można lepiej, bo możemy zauważyć, że $x_L y_R + x_R y_L = (x_L - x_R) \cdot (y_R - y_L) + x_L y_L + x_R y_R$, więc potrzebujemy policzyć tylko 3 iloczyny: $x_L y_L, x_R y_R, (x_L - x_R) \cdot (y_R - y_L)$. To daje $T(n) = 3 \cdot T\left(\frac{n}{2}\right) + \Theta(n)$

Twierdzenie 1.2. *CLRS - Uniwersalne Twierdzenie o Rekurencji* Niech $a \geq 1, b > 1$ będą stałymi, $f(n)$ funkcją i $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$ (gdzie $\frac{n}{b}$ znaczy $\left\lfloor \frac{n}{b} \right\rfloor$ lub $\left\lceil \frac{n}{b} \right\rceil$). Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & f(n) = \mathcal{O}(n^{\log_b a - \epsilon}) \text{ dla pewnego } \epsilon > 0 \\ \Theta(n^{\log_b a} \log n) & f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & f(n) = \Omega(n^{\log_b a + \epsilon}) \text{ dla pewnego } \epsilon > 0 \\ & a \cdot f(n/b) \leq c \cdot f(n) \text{ dla pewnej stałej } c < 1 \text{ dla dużych } n \end{cases}$$

Lemat 1.3. Niech $a \geq 1, b > 1$ będą stałymi i niech $f(n)$ będzie nieujemną funkcją zdefiniowaną dla potęg b . Jeśli

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & n = b^i, i \geq 1 \end{cases}$$

to

$$T(n) = \Theta(n^{\log_b n}) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

Dowód.

$$a^{\log_b n} = b^{\log_b n \cdot \log_b a} = b^{\log_b a \cdot \log_b n} = (b^{\log_b a})^{\log_b n} = n^{\log_b a} \quad (1)$$

$$\begin{aligned} T(n) &= f(n) + a \cdot T\left(\frac{n}{b}\right) = f(n) + a \left(f\left(\frac{n}{b}\right) + aT\left(\frac{n}{b^2}\right) \right) \\ &= f(n) + af\left(\frac{n}{b}\right) + a^2T\left(\frac{n}{b^2}\right) = f(n) + af\left(\frac{n}{b}\right) + a^2 \left(f\left(\frac{n}{b^2}\right) + aT\left(\frac{n}{b^3}\right) \right) \\ &= f(n) + af\left(\frac{n}{b}\right) + a^2f\left(\frac{n}{b^2}\right) + \dots + a^{\log_b n - 1} f\left(\frac{n}{b^{\log_b n}}\right) + a^{\log_b n} T(1) \\ &\stackrel{1}{=} \dots + a^{\log_b n} T(1) = \dots + n^{\log_b a} T(1) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) \end{aligned}$$

□

Przeprowadzimy teraz dowód uniwersalnego twierdzenia o rekurencji (1.2). Ograniczymy się do przypadku, gdy n jest potęgą b . Pozostały przypadek jest bardziej techniczny.

Dowód. Rozważmy wszystkie przypadki:

1. $f(n) = \mathcal{O}(n^{\log_b a - \epsilon})$. Wtedy

$$f\left(\frac{n}{b^j}\right) = \mathcal{O}\left(\left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right)$$

$$\begin{aligned} \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) &= \sum_{j=0}^{\log_b n - 1} a^j \mathcal{O}\left(\left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right) = \mathcal{O}\left(\underbrace{\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}}_{\blacktriangle}\right) \\ \blacktriangle &= \sum_{j=0}^{\log_b n - 1} a^j \frac{n^{\log_b a - \epsilon}}{b^{j(\log_b a - \epsilon)}} = n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} a^j \frac{b^{j\epsilon}}{b^{j \log_b a}} \\ &= n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} \left(\frac{a \cdot b^\epsilon}{\underbrace{b^{\log_b a}}_{=a}}\right)^j = n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} b^{\epsilon j} \quad (\text{wzór na ciąg geometryczny}) \\ &= n^{\log_b a - \epsilon} \frac{b^{\epsilon \log_b n} - 1}{b^\epsilon - 1} = n^{\log_b a - \epsilon} \frac{n^\epsilon - 1}{\underbrace{b^\epsilon - 1}_{\text{stała}}} \\ &= n^{\log_b a - \epsilon} \mathcal{O}(n^\epsilon - 1) = \mathcal{O}(n^{\log_b a - \epsilon} \cdot n^\epsilon - 1) = \mathcal{O}(n^{\log_b a}) \end{aligned}$$

$$\text{Z lematu: } T(n) = \sum_{j=0}^{\log_b n - 1} a^j T\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \mathcal{O}(n^{\log_b a}) + \Theta(n^{\log_b a}) = \Theta(n^{\log_b a})$$

2. $f(n) = \Theta(n^{\log_b a})$. Wtedy:

$$\begin{aligned}
 f\left(\frac{n}{b^j}\right) &= \Theta\left(\left(\frac{n}{b^j}\right)^{\log_b a}\right) \\
 \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) &= \sum_{j=0}^{\log_b n-1} a^j \Theta\left(\left(\frac{n}{b^j}\right)^{\log_b a}\right) = \Theta\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right) \\
 &= \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} a^j \frac{1}{b^{j \log_b a}}\right) = \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} a^j \frac{1}{\underbrace{b^{\log_b a^j}}_{=a^j}}\right) \\
 &= \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} 1\right) = \Theta(n^{\log_b a} \log_b n) \\
 \text{Z lematu: } T(n) &= \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \mathcal{O}(n^{\log_b a} \log n) + \Theta(n^{\log_b a}) = \Theta(n^{\log_b a} \log n)
 \end{aligned}$$

3. $f(n) = \Omega(n^{\log_b a + \epsilon})$ oraz $a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$ dla jakiegoś $c < 1$ i odpowiednio dużych n .

Z założenia $f\left(\frac{n}{b}\right) \leq \frac{c}{a} f(n)$, więc $f\left(\frac{n}{b^j}\right) \leq \left(\frac{c}{a}\right)^j f(n) = \frac{c^j}{a^j} f(n)$.

$$\begin{aligned}
 \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) &\leq \sum_{j=0}^{\log_b n-1} \cancel{a^j} \frac{c^j}{\cancel{a^j}} f(n) + \mathcal{O}(1) \leq f(n) \sum_{j=0}^{\log_b n-1} c^j + \mathcal{O}(1) \\
 &\leq f(n) \sum_{j=0}^{\infty} c^j + \mathcal{O}(1) \leq f(n) \frac{1}{1-c} + \mathcal{O}(1) = \mathcal{O}(f(n))
 \end{aligned}$$

$$\text{Z drugiej strony: } \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) = a^0 f\left(\frac{n}{b^0}\right) + \underbrace{\sum_{j=1}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right)}_{\geq 0} = \Omega(f(n))$$

$$\text{Z lematu: } T(n) = \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \Theta(f(n)) + \Theta(n^{\log_b a}) = \Theta(f(n))$$

□

Wniosek 1.4. Niech $a \geq 1$, $b > 1$ będą stałymi, a $f(n)$ funkcją taką, że $f(n) = \Theta(n^k)$. Niech $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$ (gdzie $\frac{n}{b}$ znaczy $\left\lfloor \frac{n}{b} \right\rfloor$ lub $\left\lceil \frac{n}{b} \right\rceil$). Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & a > b^k \Leftrightarrow \log_b a > k \\ \Theta(n^{\log_b a} \log n) & a = b^k \Leftrightarrow \log_b a = k \\ \Theta(n^k) & a < b^k \Leftrightarrow \log_b a < k \end{cases}$$

Wróćmy teraz do naszego problemu mnożenia liczb n -cyfrowych. Drugi algorytm ma złożoność:

$$T(n) = 4 \cdot T\left(\frac{n}{2}\right) + \Theta(n^1) \quad a = 4, b = 2, k = 1$$

$$4 > 2^1 \Rightarrow T(n) = \Theta(n^{\log_2 4}) = \Theta(n^2)$$

Trzeci algorytm ma złożoność:

$$T(n) = 3 \cdot T\left(\frac{n}{2}\right) + \Theta(n^1) \quad a = 3, b = 2, k = 1$$

$$3 > 2^1 \Rightarrow T(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.59})$$

Wykład 3

Przykład 2.1. Mnożenie macierzy

A, B - macierze $n \times n$

$$A \cdot B = C = ?$$

Najpopularniejszy sposób mnożenia dwóch macierzy $n \times n$ działa w czasie $\mathcal{O}(n^3)$. Musimy obliczyć n^2 wartości i obliczenie każdej zajmuje liniowy czas.

$$B = \begin{bmatrix} \vdots \end{bmatrix} \quad A = \begin{bmatrix} \dots \end{bmatrix} \begin{bmatrix} \times \end{bmatrix} = C$$

Algorytm Strassera

Zakładamy, że n jest potęgą 2 (dla przejrzystości). Dzielimy A i B na 4 macierze $\frac{n}{2} \times \frac{n}{2}$:

$$A = \left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right], \quad B = \left[\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right]$$

$$A \cdot B = \left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right] \cdot \left[\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right] = \left[\begin{array}{c|c} C_{11} & C_{12} \\ \hline C_{21} & C_{22} \end{array} \right]$$

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

Zamieniliśmy 1 mnożenie macierzy $n \times n$ na 8 mnożeń macierzy $\frac{n}{2} \times \frac{n}{2}$ + podział i dodawanie ($\mathcal{O}(n^2)$). Stąd mamy zależność rekurencyjną:

$$T(n) = 8 \cdot T\left(\frac{n}{2}\right) + \Theta(n^2) \quad a = 8, b = 2, k = 2$$

$$8 > 2^2 \Rightarrow T(n) = \Theta(n^{\log_2 8}) = \Theta(n^3)$$

Strassen zauważył, że jeśli obliczymy

$$M_1 = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$$

$$M_2 = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$

$$M_3 = (A_{11} - A_{21}) \cdot (B_{11} + B_{12})$$

$$M_4 = (A_{11} + A_{12}) \cdot B_{22}$$

$$M_5 = A_{11} \cdot (B_{12} - B_{22})$$

$$M_6 = A_{22} \cdot (B_{21} - B_{11})$$

$$M_7 = (A_{21} + A_{22}) \cdot B_{11}$$

$$\text{to } C_{11} = M_1 + M_2 - M_4 + M_6$$

$$C_{12} = M_4 + M_5$$

$$C_{21} = M_6 + M_7$$

$$C_{22} = M_2 - M_3 + M_5 - M_7$$

Zatem

$$T(n) = 7 \cdot T\left(\frac{n}{2}\right) + \Theta(n^2)$$

$$a = 7, \quad b = 2, \quad k = 2$$

$$7 > 2^2 \Rightarrow T(n) = \Theta(n^{\log_2 7}) = \Theta(n^{2.81})$$

Własność optymalnej podstruktury

Optymalne rozwiązanie problemu jest funkcją optymalnych rozwiązań podproblemów (optymalne rozwiązanie można skonstruować z optymalnych rozwiązań podproblemów). Żeby metoda *dziel i zwyciężaj* działała dobrze muszą być spełnione dwa warunki (przez problem i algorytm):

1. własność optymalnej podstruktury
2. podproblemy są rozłączne

Znajdywanie pary najbliższych punktów

Q - zbiór $n \geq 2$ punktów na płaszczyźnie

$$p_1 = (x_1, y_1) \quad p_2 = (x_2, y_2) \quad d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Problem: znaleźć parę punktów w najmniejszej odległości. Algorytm siłowy (*brute force*), który sprawdza wszystkie pary punktów działa w czasie $\Theta(n^2)$, bo mamy $\binom{n}{2} = \frac{n(n-1)}{2} = \Theta(n^2)$ par punktów. Pokażemy algorytm typu *dziel i zwyciężaj*, który działa w czasie $\mathcal{O}(n \log n)$.

W każdym wywołaniu rekurencyjnym wejście składa się z 3 rzeczy: zbioru $P \subset Q$ i dwóch tablic X i Y które zawierają wszystkie punkty z P . W tablicy X punkty są posortowane w porządku niemalejącym względem pierwszej współrzędnej, a w tablicy Y po drugiej współrzędnej.

Opis algorytmu:

jeśli $|P| \leq 3$ sprawdzamy wszystkie pary

jeśli $|P| > 3$

Dziel: Dzielimy punkty P pionową prostą l na dwa podzbiory P_L i P_R na lewo i na prawo od l tak, że $|P_L| = \left\lceil \frac{|P|}{2} \right\rceil$, $|P_R| = \left\lfloor \frac{|P|}{2} \right\rfloor$. Dzielimy tablicę X na X_L i X_R zawierające punkty odpowiednio P_L i P_R posortowane w porządku niemalejącym względem pierwszej współrzędnej. Podobnie dzielimy Y na Y_L i Y_R zawierające odpowiednio punkty z P_L i P_R posortowane w porządku niemalejącym po drugiej współrzędnej.

Zwyciężaj: Wywołujemy rekurencyjnie algorytm dla wejścia P_L , X_L , Y_L i P_R , X_R , Y_R . Algorytm znajduje pary najbliższych punktów w P_L i P_R . Niech δ_L i δ_R będą najmniejszymi odległościami znalezionymi przez algorytm dla odpowiednio P_L i P_R . Niech δ będzie mniejszą z nich: $\delta = \min(\delta_L, \delta_R)$.

Połącz: Para najbliższych punktów z P składa się z dwóch punktów z P_L lub z dwóch punktów z P_R lub jednego punktu z P_L i jednego z P_R .

Musimy sprawdzić czy istnieje para 3-go typu w odległości mniejszej niż δ . Jeśli taka para istnieje, wtedy jej punkty są w pionowym pasie o szerokości 2δ wokół l .

W celu sprawdzenia czy taka para istnieje wykonujemy następujące czynności:

1. Tworzymy tablicę Y' otrzymaną z Y poprzez usunięcie punktów spoza pasa. Y' jest posortowany w porządku niemalejącym względem drugiej współrzędnej.

2. Dla każdego punktu p z Y' sprawdzamy odległość między p , a p' dla 7 kolejnych punktów p' z Y' . Algorytm oblicza najmniejszą odległość δ' i zapamiętuje ją i odpowiednią parę punktów.
3. Jeśli $\delta' < \delta$ to pionowy pas zawiera parę punktów w mniejszej odległości niż pary znalezione w wywołaniach rekurencyjnych. Algorytm zwraca δ' i odpowiednią parę punktów. Wpp. zwraca δ i odpowiednią parę punktów.

Dowód poprawności. Jedyną rzeczą, która nie jest oczywista jest fakt, że sprawdzamy tylko 7 następnych punktów po p w Y' w fazie łącz. Przypuśćmy, że p, q są parą punktów w najmniejszej odległości w P i $\delta(p, q) = \delta'$. Niech $p = (x_p, y_p)$, $q = (x_q, y_q)$. Bez straty ogólności załóżmy $y_p \leq y_q$. Pokażemy, że algorytm znajdzie tę parę punktów. Przypuśćmy, że nie. Wtedy istnieje co najmniej 7 punktów pomiędzy p i q w Y' . Niech $r = (x_r, y_r)$ będzie jednym z nich. Skoro $d(p, q) < \delta$ to $y_q - y_p = |y_q - y_p| \leq \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2} = \delta(p, q) \leq \delta$. Ponadto y_r musi być między y_p i y_q : $y_p \leq y_r \leq y_q$, więc $y_r - y_p \leq y_q - y_p < \delta$ zatem co najmniej 9 punktów z P w prostokącie $2\delta \times \delta$ ograniczonym poziomymi liniami $y = y_p$, $y = y_p + \delta$.

Wtedy w lewym lub prawym kwadracie $\delta \times \delta$ jest przynajmniej 5 punktów z P . Przypuśćmy, że to lewy kwadrat. Zatem istnieje kwadrat $\delta \times \delta$ z 5 punktami z P w P_L . Ale wtedy istnieją dwa punkty w P_L w odległości mniejszej niż δ , bo nie da się umieścić w kwadracie $\delta \times \delta$ 5 punktów dla których odległość między dowolnymi dwoma jest mniejsza niż δ . Sprzeczność (bo najmniejsza odległość w P_L to $\delta_L \geq \delta$).

Zatem wystarczy sprawdzić tylko 7 następnych punktów po p w Y' . □

Analiza złożoności czasowej Na początku musimy posortować tablicę X i Y . To zajmuje $\mathcal{O}(n \log n)$. Jeśli X i Y są już posortowane to podział X na posortowane X_L i X_R oraz Y analogicznie zajmie $\mathcal{O}(n)$.

Algorytm 2: ???

```

1: length( $Y_L$ )  $\leftarrow$  length( $Y_R$ )  $\leftarrow$  0
2: for  $i \leftarrow 1$  to length( $Y$ ) do
3:   if  $Y(i) \in P_L$  then
4:     length( $Y_L$ )  $\leftarrow$  length( $Y_L$ ) + 1
5:      $Y_L(\text{length}(Y_L)) \leftarrow Y(i)$ 
6:   else
7:     length( $Y_R$ )  $\leftarrow$  length( $Y_R$ ) + 1
8:      $Y_R(\text{length}(Y_R)) \leftarrow Y(i)$ 

```

Mamy dwa wywołania rekurencyjne dla $n/2$ punktów i fazę łącz. Tablicę Y' można otrzymać z Y w czasie $\mathcal{O}(n)$ tak, że Y' jest posortowana. Niech $x = x_L$ będzie równanie prostej l .

Algorytm 3: ???

```

1: length( $Y'$ )  $\leftarrow$  0
2: for  $i \leftarrow 1$  to length( $Y$ ) do
3:   if  $x_L - \delta \leq Y(i).x \leq x_L + \delta$  then
4:     length( $Y'$ )  $\leftarrow$  length( $Y'$ ) + 1
5:      $Y'(\text{length}(Y')) \leftarrow Y(i)$ 

```

Skoro dla każdego z $\mathcal{O}(n)$ punktów z Y' sprawdzamy odległość tylko co najwyżej 7 punktów, to najmniejsza odległość δ' punktów z Y' znajdziemy w czasie $\mathcal{O}(n)$.

Oznaczmy złożoność algorytmu bez wstępnego sortowania przez $T(n)$. Mamy:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \Theta(n^1) \qquad a = 2, \quad b = 2, \quad k = 1$$

$$2 = 2^1 \Rightarrow T(n) = \Theta(n \log n)$$

Wykład 4 i 5

Programowanie dynamiczne

Czasami gdy zaimplementujemy jakąś formę rekurencyjną dostajemy algorytm działający w czasie wykładniczym. Powodem może być to, że jakieś wartości obliczane są wielokrotnie.

Problem liczb Fibonacciego

$$\begin{cases} F_0 = F_1 = 1 & n = 0, 1 \\ F_n = F_{n-1} + F_{n-2} & n \geq 2 \end{cases} \qquad \text{Zadanie: Obliczyć } n\text{-tą liczbę Fibonacciego}$$

Algorytm 4: Algorytm naiwny Fib1

```
function FIB1(n)
1:   if n ≤ 1 then return 1
2:   return FIB1(n - 1) + FIB1(n - 2)
```

Zobaczmy jak jest obliczane np. $\text{FIB1}(5)$. W drzewie wywołań poszczególne wartości powtarzają się:

$F(i)$ jest obliczane	razy
(1)	5
(2)	3
(3)	2
(4)	1
(5)	1

Niech $s(n)$ będzie liczbą dodawań wykonanych przez alg. w wywołaniu $\text{FIB1}(n)$.

$$\begin{cases} s(0) = s(1) = 0 \\ s(n) = s(n-1) + s(n-2) + 1 & n \geq 2 \end{cases}$$

Niech $f(n) = s(n) + 1$

Wtedy $f(0) = f(1) = 1$

$$\begin{aligned} f(n) &= s(n) + 1 = s(n-1) + s(n-2) + 1 + 1 = f(n-1) - 1 + f(n-2) - 1 + 1 + 1 \\ &= f(n-1) + f(n-2) \end{aligned}$$

Zatem $f(n) = F_n$ $f(n) = \Theta(F_n) = \Theta\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right) = \Omega(1.61^n)$. Zatem $s(n) = \Omega(1.61^n)$.

FIB1 jest algorytmem wykładniczym.

Oto alg. progr. dynamicznego:

Algorytm 5: Algorytm dynamiczny Fib2

<pre> function FIB2(n) 1: if $n \leq 1$ then return 1 2: $F(0) \leftarrow F(1) \leftarrow 1$ 3: for $i \leftarrow 2$ to n do 4: $F(i) \leftarrow F(i-1) + F(i-2)$ 5: return $F(n)$ </pre>	$\triangleright F$ - tablica
---	------------------------------

Liczba dodawań wykonanych przez FIB2 w wywołaniu FIB2(n) wynosi $n - 1$. Złożoność czasowa wynosi $\Theta(n)$.

Algorytm mnożenia macierzy

Algorytm 6: Naiwne mnożenie macierzy

<pre> function MATRIXMULTIPLY(A, B) 1: for $i \leftarrow 1$ to p do 2: for $j \leftarrow 1$ to r do 3: $C(i, j) \leftarrow 0$ 4: for $k \leftarrow 1$ to q do 5: $C(i, j) \leftarrow C(i, j) + A(i, k) \cdot B(k, j)$ 6: return C </pre>	$\triangleright A$ - macierz $p \times q$; B - macierz $q \times r$; C - macierz $p \times r$ $\triangleright$ operacja dominująca
---	---

Operacją dominującą jest mnożenie w linii 5. Mamy $p \cdot q \cdot r$ mnożeń. Złożoność jest $\mathcal{O}(p \cdot q \cdot r)$.

Problem mnożenia ciągu macierzy

Przykład: (A_1, A_2, A_3) gdzie A_1 - macierz 5×10 , A_2 - macierz 10×3 , A_3 - macierz 3×2 . Ile potrzeba mnożeń skalarnych (mnożenia liczb) do obliczenia iloczynu $A_1 \cdot A_2 \cdot A_3$?

$$A_1 \cdot A_2 \cdot A_3 = (A_1 \cdot A_2) \cdot A_3 = A_1 \cdot (A_2 \cdot A_3)$$

I sposób: $(A_1 \cdot A_2) \cdot A_3 = 5 \cdot 10 \cdot 3 + 5 \cdot 3 \cdot 2 = 150 + 30 = 180$

II sposób: $A_1 \cdot (A_2 \cdot A_3) = 10 \cdot 3 \cdot 2 + 5 \cdot 10 \cdot 2 = 60 + 100 = 160$

II sposób jest szybszy.

Problem: Dla danego ciągu macierzy ($A_1, A_2, \dots, A_n$), gdzie A_i jest macierzą $p_{i-1} \times p_i$, znajdź kolejność wykonywania mnożeń minimalizującą liczbę mnożeń skalarnych użytych do obliczenia iloczynu $A_1 \cdot A_2 \cdot \dots \cdot A_n$.

Ile jest różnych sposobów obliczenia tego iloczynu? Innymi słowy, na ile sposobów możemy poprawnie wstawić nawiasy w iloczynie $A_1 \cdot A_2 \cdot \dots \cdot A_n$?

Przykład: $A_1 \cdot A_2 \cdot A_3 \cdot A_4$ ($n = 4$)

- $((A_1 \cdot A_2) \cdot A_3) \cdot A_4$
- $(A_1 \cdot (A_2 \cdot A_3)) \cdot A_4$

- $A_1 \cdot ((A_2 \cdot A_3) \cdot A_4)$
- $A_1 \cdot (A_2 \cdot (A_3 \cdot A_4))$
- $(A_1 \cdot A_2) \cdot (A_3 \cdot A_4)$

Zatem mamy 5 sposobów.

Sprawdźmy czy umiemy rozwiązać ten problem szybko poprzez sprawdzenie wszystkich możliwości. Niech $P(n)$ – liczba sposobów wstawienia nawiasów w iloczynie n macierzy. Załóżmy, że ostatnie mnożenie w ciągu $A_1 \dots A_n$ to mnożenie między A_k i A_{k+1} : $(A_1 \dots A_k)(A_{k+1} \dots A_n)$. W podciągach lewym i prawym nawiasy są rozmieszczane niezależnie od siebie.

Stąd rekurencja:

$$\begin{cases} P(n) = \sum_{k=1}^{n-1} P(k) \cdot P(n-k) \\ P(1) = 1 \end{cases}$$

Jest to znana formuła rekurencyjna a rozwiązanie to przesunięta liczba Catalana:

$$P(n) = C(n-1), \text{ gdzie } C(n) = \frac{1}{n+1} \binom{2n}{n} \text{ jest } n\text{-tą liczbą Catalana.}$$

Wiadomo, że $C(n) = \Omega\left(\frac{4^n}{n^{3/2}}\right)$. Zatem $P(n)$ jest wykładniczy względem n . Chcemy znaleźć szybki algorytm.

Rozwiązanie z optymalną podstrukturą: Zauważmy że jeśli optymalne rozwiązanie jest postaci $(A_1 \dots A_k)(A_{k+1} \dots A_n)$ dla jakiegoś $k \in \{1, \dots, n-1\}$, to w podciągach $A_1 \dots A_k$ i $A_{k+1} \dots A_n$ rozwiązanie nawiasów musi być optymalne. Wpp. lepsze rozmieszczenie nawiasów w $A_1 \dots A_k$ lub $A_{k+1} \dots A_n$ dałoby lepsze rozmieszczenie nawiasów w całym ciągu.

Rozwiązanie optymalne zawiera rozwiązania optymalne podproblemów (własność optymalnej podstruktury). Podproblemami w naszym problemie są problemy optymalnego rozmieszczenia nawiasów w ciągach $A_i A_{i+1} \dots A_j$ dla $1 \leq i \leq j \leq n$.

Niech $m(i, j)$ oznacza minimalną liczbę mnożeń skalarnych potrzebnych do obliczenia tego iloczynu $A_i \dots A_j$ dla $1 \leq i \leq j \leq n$. Chcemy obliczyć $m(1, n)$.

$$m(i, i) = 0$$

Optymalny sposób rozmieszczenia nawiasów jest postaci $(A_i \dots A_k)(A_{k+1} \dots A_j)$ dla pewnego $k \in \{i, \dots, j-1\}$. Stąd mamy zależność:

$$m(i, j) = \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1} \cdot p_k \cdot p_j\} \quad \text{dla } i < j$$

Niech $s(i, j)$ będzie wartością k , dla którego to minimum jest osiągnięte (czyli $s(i, j)$ wskazuje miejsce ostatniego mnożenia w optymalnym rozwiązaniu podproblemu $A_i \dots A_j$). Czyli dla $1 \leq i < j \leq n$.

Jeśli zaimplementujemy po prostu tę formułę rekurencyjną otrzymamy alg. wykładniczy. Ale jest tylko $n + \binom{n}{2} = \mathcal{O}(n^2)$ podproblemów, więc rzędu $\mathcal{O}(n^2)$ liczb $m(i, j)$ i każdą z tych liczb wystarczy obliczyć jeden raz.

Algorytm 7: Algorytm Matrix Chain Order

```

function MATRIXCHAINORDER( $p$ )    ▷ ( $p_0, p_1, \dots, p_n$ ) - ciąg rozmiarów;  $A_i$  - macierz  $p_{i-1} \times p_i$ 
1:    $n \leftarrow \text{length}(p) - 1$ 
2:   for  $i \leftarrow 1$  to  $n$  do
3:      $m(i, i) \leftarrow 0$ 
4:     for  $l \leftarrow 2$  to  $n$  do                                ▷  $l$  - liczba macierzy w podproblemie
5:       for  $i \leftarrow 1$  to  $n - l + 1$  do                        ▷  $l = 2$  :  $A_1 A_2, A_2 A_3, \dots, A_{n-1} A_n$ 
6:          $j \leftarrow i + l - 1$                                 ▷  $l = 3$  :  $A_1 A_2 A_3, A_2 A_3 A_4, \dots, A_{n-2} A_{n-1} A_n$ 
7:          $m(i, j) \leftarrow \infty$                                 ▷  $l = n$  :  $A_1 \dots A_n$ 
8:         for  $k \leftarrow i$  to  $j - 1$  do
9:            $q \leftarrow m(i, k) + m(k + 1, j) + p_{i-1} \cdot p_k \cdot p_j$ 
10:        if  $q < m(i, j)$  then
11:           $m(i, j) \leftarrow q$ 
12:           $s(i, j) \leftarrow k$ 
13:   return  $m, s$ 

```

Mamy 3 pętle (po l, i, k) i liczbę iteracji każdej z tych pętli można ograniczyć przez n , zatem złożoność czasowa to $\mathcal{O}(n^3)$.

Przykład: $n = 5$, $p = (4, 5, 2, 1, 2, 3)$.

$A_1 - 4 \times 5$, $A_2 - 5 \times 2$, $A_3 - 2 \times 1$, $A_4 - 1 \times 2$, $A_5 - 2 \times 3$.

$$m \left[\begin{array}{c|c|c|c|c|c} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & 0 & 40 & 30 & 38 & 48 \\ \hline 2 & \times & 0 & 10 & 20 & 31 \\ \hline 3 & \times & \times & 0 & 4 & 12 \\ \hline 4 & \times & \times & \times & 0 & 6 \\ \hline 5 & \times & \times & \times & \times & 0 \end{array} \right]$$

$$s \left[\begin{array}{c|c|c|c|c|c} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & \times & 1 & 1 & 3 & 3 \\ \hline 2 & \times & \times & 2 & 3 & 3 \\ \hline 3 & \times & \times & \times & 3 & 3 \\ \hline 4 & \times & \times & \times & \times & 4 \\ \hline 5 & \times & \times & \times & \times & \times \end{array} \right]$$

$$\begin{aligned}
 L = 2 : \quad & m(1, 2) = 4 \cdot 5 \cdot 2 = 40 \\
 & m(2, 3) = 5 \cdot 2 \cdot 1 = 10 \\
 & m(3, 4) = 2 \cdot 1 \cdot 2 = 4 \\
 & m(4, 5) = 1 \cdot 2 \cdot 3 = 6
 \end{aligned}$$

$$\begin{aligned}
 L = 3 : \quad & m(1, 3) = \min \begin{cases} m(1, 1) + m(2, 3) + 4 \cdot 5 \cdot 1 \\ m(1, 2) + m(3, 3) + 4 \cdot 2 \cdot 1 \end{cases} = \min \begin{cases} 0 + 10 + 20 \\ 40 + 0 + 8 \end{cases} = \min \begin{cases} 30 \\ 48 \end{cases} = 30 \\
 & m(2, 4) = \min \begin{cases} m(2, 2) + m(3, 4) + 5 \cdot 2 \cdot 2 \\ m(2, 3) + m(4, 4) + 5 \cdot 1 \cdot 2 \end{cases} = \min \begin{cases} 0 + 4 + 20 \\ 10 + 0 + 10 \end{cases} = \min \begin{cases} 24 \\ 20 \end{cases} = 20 \\
 & m(3, 5) = \min \begin{cases} m(3, 3) + m(4, 5) + 2 \cdot 1 \cdot 3 \\ m(3, 4) + m(5, 5) + 2 \cdot 2 \cdot 3 \end{cases} = \min \begin{cases} 0 + 6 + 6 \\ 4 + 0 + 12 \end{cases} = \min \begin{cases} 12 \\ 16 \end{cases} = 12
 \end{aligned}$$

$$\begin{aligned}
 L = 4 : \quad & m(1, 4) = \min \begin{cases} m(1, 1) + m(2, 4) + 4 \cdot 5 \cdot 2 \\ m(1, 2) + m(3, 4) + 4 \cdot 2 \cdot 2 \\ m(1, 3) + m(4, 4) + 4 \cdot 1 \cdot 2 \end{cases} = \min \begin{cases} 0 + 20 + 40 \\ 40 + 4 + 16 \\ 30 + 0 + 8 \end{cases} = \min \begin{cases} 60 \\ 60 \\ 38 \end{cases} = 38 \\
 & m(2, 5) = \min \begin{cases} m(2, 2) + m(3, 5) + 5 \cdot 2 \cdot 3 \\ m(2, 3) + m(4, 5) + 5 \cdot 1 \cdot 3 \\ m(2, 4) + m(5, 5) + 5 \cdot 2 \cdot 3 \end{cases} = \min \begin{cases} 0 + 12 + 30 \\ 10 + 6 + 15 \\ 20 + 0 + 30 \end{cases} = \min \begin{cases} 42 \\ 31 \\ 50 \end{cases} = 31
 \end{aligned}$$

$$L = 5 : \quad m(1, 5) = \min \begin{cases} m(1, 1) + m(2, 5) + 4 \cdot 5 \cdot 3 \\ m(1, 2) + m(3, 5) + 4 \cdot 2 \cdot 5 \\ m(1, 3) + m(4, 5) + 4 \cdot 1 \cdot 3 \\ m(1, 4) + m(5, 5) + 4 \cdot 2 \cdot 3 \end{cases} = \min \begin{cases} 0 + 31 + 60 \\ 40 + 12 + 24 \\ 30 + 6 + 12 \\ 38 + 0 + 24 \end{cases} = \min \begin{cases} 91 \\ 76 \\ 48 \\ 62 \end{cases} = 48$$

Oto algorytm znajdowania optymalnego rozwiązania:

Algorytm 8: Algorytm Matrix Chain Multiply

function MATRIXCHAINMULTIPLY(A, s, i, j)	$\triangleright A = (A_1, \dots, A_n)$
1: if $j = i$ then	
2: return A_i	
3: $X \leftarrow$ MATRIXCHAINMULTIPLY($A, s, i, s(i, j)$)	
4: $Y \leftarrow$ MATRIXCHAINMULTIPLY($A, s, s(i, j) + 1, j$)	
5: return MATRIXMULTIPLY(X, Y)	

W celu znalezienia rozwiązania wywołujemy: MATRIXCHAINMULTIPLY($A, s, 1, n$)

$$A_1 \cdot A_2 \cdot A_3 \cdot A_4 \cdot A_5$$

$$s(1, 5) = 3 \implies (A_1 \cdot A_2 \cdot A_3)(A_4 \cdot A_5)$$

$$s(1, 3) = 1 \implies (A_1 \cdot (A_2 \cdot A_3))(A_4 \cdot A_5) \quad s(4, 5) = 4 \implies \text{bez zmian}$$

$$s(2, 3) = 2 \implies (A_1 \cdot (A_2 \cdot A_3))(A_4 \cdot A_5)$$

Aby algorytm zadziałał dobrze, problem musi spełniać warunki:

1. **Własność optymalnej podstruktury** – optymalne rozwiązanie problemu można skonstruować z optymalnych rozwiązań podproblemów.
2. **Własność wspólnych podproblemów** – sytuacja, gdy prosty algorytm rekurencyjny obliczałby rozwiązania tych samych podproblemów wielokrotnie. Liczba podproblemów powinna być „mała”.

W klasycznej metodzie programowania dynamicznego rozwiązanie jest budowane od dołu do góry – od podproblemów najmniejszych do największych. Jest też podejście, w którym rozwiązanie jest budowane od góry do dołu, podobnie jak w prostym algorytmie rekurencyjnym, ale z tą różnicą, że unikamy obliczania tych samych podproblemów wielokrotnie. To podejście nazywa się **spamiętywaniem** (*memoization*).

Problem mnożenia ciągu macierzy ze spamiętywaniem Algorytm oblicza tablice m i s jak poprzednio, ale w inny sposób. Dopóki podproblem nie jest rozwiązany, $m(i, j)$ przechowuje ∞ . Po rozwiązaniu podproblemu (i, j) umieszczamy w $m(i, j)$ odpowiednią wartość i od tej pory używamy jej bez obliczania.

Algorytm 9: Algorytm Lookup Chain

```

function LOOKUPCHAIN( $p, i, j$ )       $\triangleright p = (p_0, \dots, p_n)$  - ciąg wymiarów;  $A_i$  - macierz  $p_{i-1} \times p_i$ 
1:   if  $m(i, j) < \infty$  then
2:     return  $m(i, j), s(i, j)$ 
3:   if  $i = j$  then
4:      $m(i, j) \leftarrow 0$ 
5:   else
6:     for  $k \leftarrow i$  to  $j - 1$  do
7:        $q \leftarrow \text{LOOKUPCHAIN}(p, i, k) + \text{LOOKUPCHAIN}(p, k + 1, j) + p_{i-1} \cdot p_k \cdot p_j$ 
8:       if  $q < m(i, j)$  then
9:          $m(i, j) \leftarrow q$ 
10:         $s(i, j) \leftarrow k$ 
11:  return  $(m(i, j), s(i, j))$ 

```

Algorytm 10: Algorytm Memoized Matrix Chain

```

function MEMOIZEDMATRIXCHAIN( $p$ )
1:    $n \leftarrow \text{length}(p) - 1$ 
2:   for  $i \leftarrow 1$  to  $n$  do
3:     for  $j \leftarrow i$  to  $n$  do
4:        $m(i, j) \leftarrow \infty$ 
5:   return LOOKUPCHAIN( $p, 1, n$ )

```

Złożoność czasowa:

- Linie 1–4 w MEMOIZEDMATRIXCHAIN zajmują $\Theta(n^2)$.
- Każdy z $\mathcal{O}(n^2)$ elementów tablicy m jest modyfikowany raz, później odczytanie wartości $m(i, j)$ zajmuje czas stały.
- Modyfikacja $m(i, j)$ zajmuje $\mathcal{O}(n)$ czasu (pętla w liniach 5–9 w LOOKUPCHAIN).

Ostatecznie złożoność algorytmu wynosi $\mathcal{O}(n^3)$.

Problem najdłuższego wspólnego podciągu

Niech $X = (x_1, \dots, x_m)$ – ciąg.

$Z = (z_1, \dots, z_k)$ jest podciągiem X , jeśli istnieje rosnący ciąg indeksów $1 \leq i_1 < i_2 < \dots < i_k \leq m$ takich, że dla każdego $j \in \{1, \dots, k\}$, $z_j = x_{i_j}$.

Przykład: $X = (A, B, C, B, D, A, B)$, $Z = (B, C, D, B)$. Indeksy: (2, 3, 5, 7).

Problem: Dane są ciągi $X = (x_1, \dots, x_m)$ oraz $Y = (y_1, \dots, y_n)$. Należy znaleźć najdłuższy wspólny podciąg (LCS – *Longest Common Subsequence*) X i Y , czyli najdłuższy ciąg, który jest podciągiem zarówno X , jak i Y .

Wszystkich podciągów X jest 2^m , więc algorytm sprawdzający wszystkie podciągi X działa w czasie wykładniczym.

Niech $X_i = (x_1, \dots, x_i)$ dla $i \in \{1, \dots, m\}$ i dodatkowo $X_0 = ()$ (ciąg pusty). Analogicznie dla Y .

Lemat 3.1. Niech $X = (x_1, \dots, x_m)$, $Y = (y_1, \dots, y_n)$ i niech $Z = (z_1, \dots, z_k)$ będzie LCS X i Y .

1. Jeśli $x_m = y_n$, to $z_k = x_m = y_n$ i Z_{k-1} jest LCS X_{m-1} i Y_{n-1} .
2. Jeśli $x_m \neq y_n$ i $z_k \neq x_m$, to Z jest LCS X_{m-1} i Y .
3. Jeśli $x_m \neq y_n$ i $z_k \neq y_n$, to Z jest LCS X i Y_{n-1} .

Dowód.

1. Załóżmy, że $x_m = y_n$. Gdyby $z_k \neq x_m$, to moglibyśmy wydłużyć podciąg Z przez dodanie $z_{k+1} = x_m = y_n$, co daje sprzeczność, bo $(z_1, \dots, z_{k+1})$ jest wspólnym podciągiem X i Y dłuższym niż Z . Zatem $z_k = x_m = y_n$ i Z_{k-1} jest wspólnym podciągiem X_{m-1} i Y_{n-1} . Gdyby istniał wspólny podciąg W ciągów X_{m-1} i Y_{n-1} długości $> k - 1$, to dodając na koniec W element $x_m = y_n$ otrzymalibyśmy wspólny podciąg X i Y długości $> k - 1 + 1 = k$. Sprzeczność, bo długość Z wynosi k .
2. Załóżmy, że $x_m \neq y_n$ i $z_k \neq x_m$. Wtedy Z jest wspólnym podciągiem ciągów X_{m-1} i Y . Gdyby istniał wspólny podciąg W ciągów X_{m-1} i Y o długości $> k$, to W byłby wspólnym podciągiem ciągów X i Y dłuższym niż Z . Sprzeczność.
3. Symetrycznie do punktu 2.

□

Rozwiązanie dla problemu (X, Y) można skonstruować z rozwiązań podproblemów (X_{m-1}, Y_{n-1}) , (X_{m-1}, Y) i (X, Y_{n-1}) – **własność optymalnej podstruktury**. Podproblem (X_{m-1}, Y_{n-1}) jest zawarty w podproblemach (X_{m-1}, Y) i (X, Y_{n-1}) – **własność wspólnych podproblemów**.

Niech $c(i, j)$ oznacza długość LCS ciągów X_i i Y_j .

$$c(i, j) = \begin{cases} 0 & \text{jeśli } i = 0 \text{ lub } j = 0 \\ c(i-1, j-1) + 1 & \text{jeśli } i, j > 0 \text{ i } x_i = y_j \\ \max\{c(i-1, j), c(i, j-1)\} & \text{jeśli } i, j > 0 \text{ i } x_i \neq y_j \end{cases}$$

Będziemy też używać tablicy $b(i, j)$, aby znaleźć sam podciąg LCS. Tablice mają wymiary: $c[0 \dots m, 0 \dots n]$ oraz $b[1 \dots m, 1 \dots n]$. Wartości w tablicy kierunków to $b(i, j) \in \{\nwarrow, \uparrow, \leftarrow\}$.

Algorytm 11: Algorytm obliczający długość LCS


```

function LCS( $X, Y$ )
1:   $m \leftarrow \text{length}(X)$ 
2:   $n \leftarrow \text{length}(Y)$ 
3:  for  $i \leftarrow 0$  to  $m$  do
4:     $c(i, 0) \leftarrow 0$ 
5:  for  $j \leftarrow 0$  to  $n$  do
6:     $c(0, j) \leftarrow 0$ 
7:  for  $i \leftarrow 1$  to  $m$  do
8:    for  $j \leftarrow 1$  to  $n$  do
9:      if  $x_i = y_j$  then
10:         $c(i, j) \leftarrow c(i - 1, j - 1) + 1$ 
11:         $b(i, j) \leftarrow \nwarrow$ 
12:      else if  $c(i - 1, j) \geq c(i, j - 1)$  then
13:         $c(i, j) \leftarrow c(i - 1, j)$ 
14:         $b(i, j) \leftarrow \leftarrow$ 
15:      else
16:         $c(i, j) \leftarrow c(i, j - 1)$ 
17:         $b(i, j) \leftarrow \nearrow$ 
18:  return ( $c, b$ )

```

Przykład: $X = (A, B, C, B, D, A, B)$, $Y = (B, D, C, A, B, A)$.
 $m = 7$, $n = 6$.

	j	0	1	2	3	4	5	6
i		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	$\uparrow 0$	$\uparrow 0$	$\uparrow 0$	$\nwarrow 1$	$\leftarrow 1$	$\nwarrow 1$
2	B	0	$\nwarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\uparrow 1$	$\nwarrow 2$	$\leftarrow 2$
3	C	0	$\uparrow 1$	$\uparrow 1$	$\nwarrow 2$	$\leftarrow 2$	$\uparrow 2$	$\uparrow 2$
4	B	0	$\nwarrow 1$	$\uparrow 1$	$\uparrow 2$	$\uparrow 2$	$\nwarrow 3$	$\leftarrow 3$
5	D	0	$\uparrow 1$	$\nwarrow 2$	$\uparrow 2$	$\uparrow 2$	$\uparrow 3$	$\uparrow 3$
6	A	0	$\uparrow 1$	$\uparrow 2$	$\uparrow 2$	$\nwarrow 3$	$\uparrow 3$	$\nwarrow 4$
7	B	0	$\nwarrow 1$	$\uparrow 2$	$\uparrow 2$	$\uparrow 3$	$\nwarrow 4$	$\uparrow 4$

Algorytm 12: Wypisywanie najdłuższego wspólnego podciągu

```

function PRINTLCS( $b, X, Y, i, j$ )
1:  if  $i = 0$  lub  $j = 0$  then
2:    return  $\varepsilon$ 
3:  if  $b(i, j) = \nwarrow$  then
4:    PRINTLCS( $b, X, Y, i - 1, j - 1$ )
5:    print  $x_i$ 
6:  else if  $b(i, j) = \leftarrow$  then
7:    PRINTLCS( $b, X, Y, i - 1, j$ )
8:  else
9:    PRINTLCS( $b, X, Y, i, j - 1$ )

```

▷ ciąg pusty

Żeby wypisać rozwiązanie, wywołujemy PRINTLCS(b, X, Y, m, n).
Dla powyższego przykładu otrzymamy: (B, C, B, A) .

Złożoność czasowa tego algorytmu wynosi $\Theta(m \cdot n)$.

Wykład 6

Algorytmy Zachłanne

Pomysł: Algorytm w każdym kroku dokonuje lokalnie najlepszego wyboru (najlepszego w danym momencie).

Problem wyboru zajęć $S = \{z_1, \dots, z_n\}$ – zbiór zajęć wymagający tych samych zasobów.

s_i – czas rozpoczęcia zajęcia z_i

$s_i, t_i \in \mathbb{N}, \quad s_i < t_i$

t_i – czas zakończenia zajęcia z_i

Zajęcia z_i i z_j nazywamy **zgodnymi**, jeśli $\langle s_i, t_i \rangle \cap \langle s_j, t_j \rangle = \emptyset$. Zadanie polega na znalezieniu podzbioru o największej liczności $A \subseteq S$ parami zgodnych zajęć.

i	1	2	3	4	5	6
s_i	0	2	4	7	1	7
t_i	4	6	7	10	3	9

Porządkujemy zajęcia w czasie niemalejącym względem czasu zakończenia. To zajmuje $\mathcal{O}(n \log n)$ czasu. Od tej pory zakładamy $t_1 \leq t_2 \leq \dots \leq t_n$ (zajęcia są już posortowane).

i	1	2	3	4	5	6
s_i	1	0	2	4	7	7
t_i	3	4	6	7	9	10

Podjęcie programowania dynamicznego Definiujemy zbiory zajęć:

$$S_{ij} = \{z_k \in S : t_i \leq s_k < t_k \leq s_j\}$$

(zbiór zajęć zaczynających się po czasie zakończenia zajęcia z_i i kończących się przed czasem rozpoczęcia zajęcia z_j).

Dodajemy 2 sztuczne zajęcia: z_0 z czasem $t_0 = 0$ i z_{n+1} z czasem $s_{n+1} = \infty$. Wtedy $S = S_{0,n+1}$. Jeśli $i \geq j$, to $S_{ij} = \emptyset$, bo $s_j < t_j \leq t_i$ (bo zajęcia są posortowane).

Problem: znaleźć podzbiór o największej liczności zbioru S_{ij} składający się z parami zgodnych zajęć dla $0 \leq i \leq j \leq n+1$.

Niech A_{ij} będzie jakimkolwiek podzbiorem największej liczności składającym się z parami zgodnych zajęć zbioru S_{ij} oraz niech $c(i, j) = |A_{ij}|$.

Podproblem S_{ij} : Załóżmy, że $z_k \in S_{ij}$. Jeśli wybierzemy z_k do zbioru rozwiązania, to otrzymamy dwa zbiory (podproblemy):

- S_{ik} (zbiór zajęć, które zaczynają się po zakończeniu zajęcia z_i i kończą się przed rozpoczęciem zajęcia z_k),
- S_{kj} (zbiór zajęć, które zaczynają się po zakończeniu zajęcia z_k i kończą przed rozpoczęciem zajęcia z_j).

Rozwiązanie dla S_{ij} będzie sumą rozwiązań dla S_{ik} oraz S_{kj} oraz elementu $\{z_k\}$.

Własność optymalnej podstruktury: Załóżmy, że $z_k \in A_{ij}$. Wtedy $A_{ij} = A_{ik} \cup \{z_k\} \cup A_{kj}$. Gdyby istniało rozwiązanie A'_{ik} dla S_{ik} takie, że $|A'_{ik}| > |A_{ik}|$, to zbiór $A'_{ij} = A'_{ik} \cup \{z_k\} \cup A_{kj}$ byłby rozwiązaniem dla S_{ij} takim, że $|A'_{ij}| > |A_{ij}|$. Sprzeczność. Analogicznie dla S_{kj} . Zatem A_{ik} oraz A_{kj} muszą być rozwiązaniami optymalnymi dla odpowiednio S_{ik} i S_{kj} .

Stąd wzór rekurencyjny:

$$\begin{cases} c(i, j) = 0 & \text{dla } i \geq j \\ c(i, j) = \max_{i < k < j} \{c(i, k) + c(k, j) + 1\} \end{cases}$$

Używając tego wzoru rekurencyjnego możemy skonstruować algorytm programowania dynamicznego. Mamy $\mathcal{O}(n^2)$ podproblemów. Dla $j > i$ mamy $j - i - 1$ wartości k do sprawdzenia, czyli rozwiązanie jednego podproblemu zajmie czas liniowy. Otrzymujemy algorytm o złożoności $\mathcal{O}(n^3)$.

Algorytm zachłanny

Lemat 4.1. Niech $z_m \in S_{ij}$ będzie zajęciem takim, że $t_m = \min\{t_k : z_k \in S_{ij}\}$. Wtedy:

1. Istnieje podzbiór o największej liczności składający się z parami zgodnych zajęć zbioru S_{ij} zawierający z_m .
2. $S_{im} = \emptyset$ (zatem po wybraniu z_m do zbioru rozwiązania zostaje co najwyżej 1 niepusty podproblem do rozwiązania – S_{mj}).

Dowód. Ad 2. Przypuśćmy, że nie. Niech $z_k \in S_{im}$. Wtedy $t_i \leq s_k < t_k \leq s_m < t_m \leq s_j$. Zatem $z_k \in S_{ij}$ oraz $t_k < t_m$ – sprzeczność z definicją zajęcia z_m .

Ad 1. Niech A_{ij} będzie podzbiorem o największej liczności składającym się z zajęć parami zgodnych zbioru S_{ij} . Niech z_k będzie zajęciem z A_{ij} o najmniejszym czasie zakończenia. Jeśli $z_k = z_m$, to teza jest udowodniona. Przypuśćmy, że $z_k \neq z_m$ i rozważmy zbiór $A'_{ij} = A_{ij} \setminus \{z_k\} \cup \{z_m\}$. Skoro $t_m \leq t_k$ oraz zajęcia z A_{ij} są parami zgodne, to zajęcia z A'_{ij} również są parami zgodne. Mamy $|A_{ij}| = |A'_{ij}|$, więc A'_{ij} jest rozwiązaniem optymalnym zawierającym z_m . $\square$

Z lematu wynika, że nie musimy sprawdzać $j - i - 1$ wartości k , możemy wybrać zajęcie o najmniejszym czasie zakończenia $z_m \in S_{ij}$. Żeby rozwiązać podproblem S_{ij} możemy dodać z_m do zbioru rozwiązania a następnie dodać do zbioru rozwiązania rozwiązanie optymalne podproblemu S_{mj} . Zaczynamy z $S = S_{0,n+1}$ i $t_0 = 0$. W pierwszym kroku algorytm wybiera z_1 .

Algorytm 13: Zachłanny wybór zajęć

function GREEDYACTIVITYSELECTOR(s, t)		$\triangleright$ zajęcia są posortowane po czasie zakończeń
1:	$n \leftarrow \text{length}(s)$	
2:	$A \leftarrow \{z_1\}$	$\triangleright A$ - zbiór rozwiązań
3:	$j \leftarrow 1$	
4:	for $i \leftarrow 2$ to n do	
5:	if $s_i \geq t_j$ then	
6:	$A \leftarrow A \cup \{z_i\}$	
7:	$j \leftarrow i$	
8:	return A	

Algorytm zachłanny w każdym kroku wybiera zajęcie zgodne ze wszystkimi zajęciami poprzednio wybranymi, o najmniejszym czasie zakończenia, aby zostawić jak najwięcej miejsca dla zajęć następnych (algorytm zachłanny). Skoro one są posortowane, jeśli $j < k$ i z_k jest zgodne z z_j , to z_k jest też zgodne z z_i dla każdego $1 \leq i < j$.

Analiza poprawności: Algorytm zwraca zbiór parami zgodnych zajęć, bo dodaje do zbioru rozwiązania zajęcie tylko jeśli jest zgodne ze wszystkimi zajęciami dodanymi wcześniej.

Pokażemy przez indukcję po n , że algorytm znajduje rozwiązanie optymalne.

- Dla $n = 1$ oczywiste ✓
- Niech $n > 1$. Z lematu istnieje optymalne rozwiązanie A_1 takie, że $z_1 \in A_1$.
Rozważmy problem dla $S' = \{z_i \in S : s_i \geq t_1\}$. Oczywiście zachodzi $|S'| < |S|$.
Dla każdego $z_j \in A_1 \setminus \{z_1\}$, mamy $z_j \in S'$, bo $s_j \geq t_1$, co wynika z faktu, że A_1 jest rozwiązaniem dla S .
Pokażemy, że $A_1 \setminus \{z_1\}$ jest rozwiązaniem optymalnym dla S' .
Przypuśćmy, że istnieje rozwiązanie B dla S' takie, że $|B| > |A_1 \setminus \{z_1\}|$. Wtedy $\{z_1\} \cup B$ byłoby rozwiązaniem dla S takim, że:

$$|\{z_1\} \cup B| = 1 + |B| > 1 + |A_1 \setminus \{z_1\}| = |A_1|$$

Otrzymujemy sprzeczność, bo A_1 jest z założenia rozwiązaniem optymalnym.

Z założenia indukcyjnego nasz algorytm znajduje rozwiązanie optymalne B' dla S' . Wtedy $|B'| = |A_1 \setminus \{z_1\}| = |A_1| - 1$.

Zbiór $\{z_1\} \cup B'$ jest rozwiązaniem znalezionym przez algorytm dla S i:

$$|\{z_1\} \cup B'| = 1 + |B'| = 1 + |A_1| - 1 = |A_1|$$

Więc $\{z_1\} \cup B'$ jest rozwiązaniem optymalnym.

Złożoność czasowa

- linia 1 – $\mathcal{O}(n)$
- linia 2 – $\mathcal{O}(1)$
- iteracji pętli w linii 3 jest $n - 1$
- wewnątrz pętli zajmuje czas $\mathcal{O}(1)$

Całość zajmuje $\Theta(n)$, jeśli zajęcia są już posortowane po czasie zakończenia. Ostatecznie złożoność algorytmu wynosi (uwzględniając początkowe sortowanie):

$$\mathcal{O}(n \log n) + \Theta(n) = \mathcal{O}(n \log n)$$

Wykład 7

Kody Huffmana

Często znaki są reprezentowane przy użyciu 8 bitów (np. ASCII). Skoro częstość znaków się różni, lepiej reprezentować znaki występujące często przy użyciu krótszych ciągów bitów, a znaki występujące rzadko przy użyciu dłuższych ciągów bitów.

częstość = liczba wystąpień

Przykład W pliku jest 58 znaków. Każdy znak jest elementem zbioru $\{a, e, i, s, t, r, n\}$.

		<i>a</i>	<i>e</i>	<i>i</i>	<i>s</i>	<i>t</i>	<i>r</i>	<i>n</i>
	częstość	10	15	12	3	4	13	1
kod 1	stała długość słowa kodowego	000	001	010	011	100	101	110
kod 2	zmienna długość słowa kodowego	001	01	10	00000	0001	11	00001
kod 3	zmienna długość słowa kodowego	001	01	10	00010	0001	11	00001

Jeśli użyjemy kodu o stałej długości słowa kodowego (kod 1) to zakodowany plik ma: $58 \cdot 3 = 174$ bity.

Jeśli użyjemy kodu o zmiennej długości słowa kodowego (kod 2 lub kod 3) to zakodowany plik ma:

$$10 \cdot 3 + 15 \cdot 2 + 12 \cdot 2 + 3 \cdot 5 + 4 \cdot 4 + 13 \cdot 2 + 1 \cdot 5 = 146 \text{ bitów.}$$

Jeśli używamy kodu o zmiennej długości słowa kodowego, to chcemy aby spełniony był warunek: **żadne słowo kodowe nie jest prefixem innego słowa kodowego.**

Takie kody nazywa się **kodami prefixowymi** (choć powinno być bezprefixowe).

Rozważmy kod 3. Ciąg 0001001 można odkodować na dwa sposoby:

- 00010 01 $\Rightarrow s \ e$
- 0001 001 $\Rightarrow t \ a$

Tutaj 0001 jest prefixem 00010, czyli słowo kodowe znaku *t* jest prefixem słowa kodowego znaku *s*.

Reprezentacja drzewowa kodów Słowo kodowe znaku jest ciągiem etykiet krawędzi na ścieżce od korzenia do wierzchołka zaetykietowanego tym znakiem.

Uwaga. Żadne słowo kodowe nie jest prefixem innego słowa kodowego $\iff$ wszystkie znaki są reprezentowane przez liście w reprezentacji drzewowej kodu.

Etykietujemy wierzchołki w następujący sposób: każdy wierzchołek *v* dostaje etykietę będącą sumą wystąpień znaków, które są reprezentowane przez liście poddrzewa ukorzonego w *v*.

Niech *C* będzie alfabetem, $f : C \rightarrow \mathbb{N}_+$, $f(c)$ = częstość znaku $c \in C$.

$d_T(c)$ – głębokość liścia reprezentującego znak $c \in C$ w drzewie *T*

$d_T(c)$ = długość słowa kodowego znaku $c \in C$ w kodzie reprezentowanym przez drzewo *T*

Liczba bitów w zakodowanym pliku (kod reprezentowany przez drzewo *T*) wynosi:

$$B(T) = \sum_{c \in C} f(c) \cdot d_T(c)$$

Kod prefixowy reprezentowany przez drzewo *T* nazywamy **optymalnym**, jeśli $B(T)$ jest najmniejsze możliwe.

Drzewo ukorzone nazywamy **binarnym**, jeśli każdy wierzchołek ma co najwyżej dwoje dzieci. Drzewo binarne nazywamy **regularnym**, jeśli każdy wierzchołek wewnętrzny ma dokładnie dwoje dzieci.

Twierdzenie 5.1. Drzewo binarne reprezentujące optymalny kod jest regularne.

mentować używając kopca binarnego. Wtedy koszty operacji wynoszą:

- INSERT – $\mathcal{O}(\log n)$
- EXTRACT_MIN – $\mathcal{O}(\log n)$
- CREATE – $\mathcal{O}(n)$

gdzie n jest liczbą elementów w kolejce.

Algorytm 14: Algorytm Huffmana

```

function HUFFMAN( $C, f$ )
1:   $n \leftarrow |C|$ 
2:   $Q \leftarrow \text{CREATE}(C, f)$ 
3:  for  $i \leftarrow 1$  to  $n - 1$  do
4:     $z \leftarrow \text{NEW\_NODE}$ 
5:     $x \leftarrow \text{LEFT}(z) \leftarrow \text{EXTRACT\_MIN}(Q)$ 
6:     $y \leftarrow \text{RIGHT}(z) \leftarrow \text{EXTRACT\_MIN}(Q)$ 
7:     $f(z) \leftarrow f(x) + f(y)$ 
8:    INSERT( $Q, z$ )
9:  return EXTRACT_MIN( $Q$ )
    
```

▷ Q - kolejka priorytetowa

Algorytm konstruuje drzewo binarne. z - wierzchołek, $left, right$ - dzieci.

Złożoność czasowa

- linia 1 – $\mathcal{O}(1)$
- linia 2 – $\mathcal{O}(n)$
- wewnątrz pętli z linii 3 wykona się $n - 1$ razy:
 - linia 4 – $\mathcal{O}(1)$
 - linie 5, 6 – $\mathcal{O}(\log n)$
 - linia 7 – $\mathcal{O}(1)$
 - linia 8 – $\mathcal{O}(\log n)$

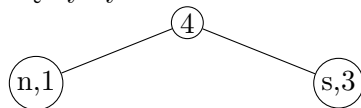
Wewnątrz pętli zajmuje łącznie czas $\mathcal{O}(\log n)$.

$$\mathcal{O}(n) + (n - 1) \cdot \mathcal{O}(\log n) = \mathcal{O}(n \log n)$$

Złożoność czasowa algorytmu wynosi $\mathcal{O}(n \log n)$.

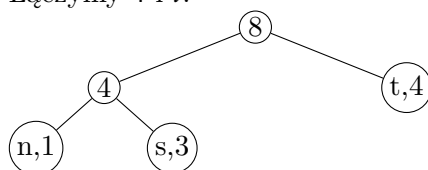
Przykład Zaczynamy ze zbiorem wierzchołków: $(n, 1), (s, 3), (t, 4), (a, 10), (i, 12), (r, 13), (e, 15)$.

1. Łączymy n i s :



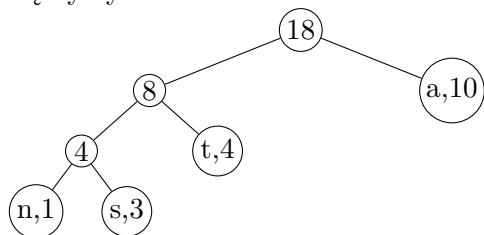
Zostają: $(t, 4), (a, 10), (i, 12), (r, 13), (e, 15)$.

2. Łączymy 4 i t :



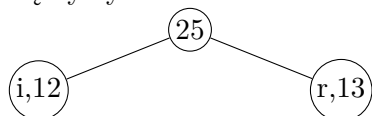
Zostają: $(a, 10), (i, 12), (r, 13), (e, 15)$.

3. Łączymy 8 i a :



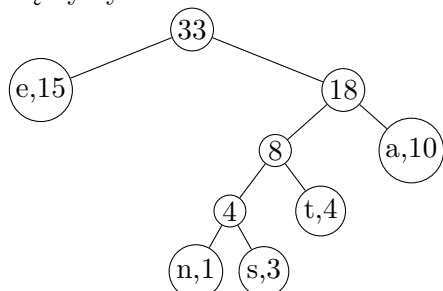
Zostają: $(i, 12), (r, 13), (e, 15)$.

4. Łączymy i i r :

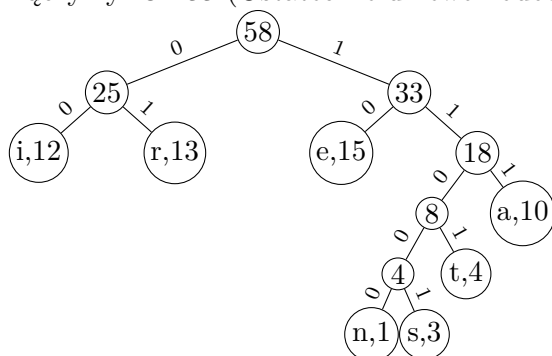


Zostaje: $(e, 15)$.

5. Łączymy e i 18:



6. Łączymy 25 i 33 (Ostateczne drzewo kodów):



Odczytany kod z drzewa w kroku 6:

$i \Rightarrow 00$
 $r \Rightarrow 01$
 $e \Rightarrow 10$
 $n \Rightarrow 11000$
 $s \Rightarrow 11001$
 $t \Rightarrow 1101$
 $a \Rightarrow 111$

Wykład 9

Matroidy

Definicja 6.1. Matroid to para $M = (S, \mathcal{A})$ taka, że:

1. S – niepusty zbiór skończony,

2. $\mathcal{A} \subseteq 2^S$ i $\mathcal{A} \neq \emptyset$ – czyli $\mathcal{A}$ to niepusta rodzina podzbiorów S ,
3. $\forall A, B \subseteq S \quad A \in \mathcal{A} \wedge B \subseteq A \implies B \in \mathcal{A}$,
4. $\forall A, B \subseteq S \quad A, B \in \mathcal{A} \wedge |B| > |A| \implies$ istnieje $x \in B \setminus A$ taki, że $A \cup \{x\} \in \mathcal{A}$ (własność wymiany).

$\mathcal{A}$ – rodzina zbiorów niezależnych matroidu M , elementy $\mathcal{A}$ – zbiory niezależne matroidu M .

Przykład 6.2.

1. A – macierz, S – zbiór wierszy A , $\mathcal{A} = \{R \subseteq S : R \text{ tworzy zbiór liniowo niezależny}\}$.
Wtedy $(S, \mathcal{A})$ to matroid.
2. $G = (V, E)$ – graf, $\mathcal{A} = \{F \subseteq E : G[F] \text{ jest acykliczny}\}$.
Wtedy $(E, \mathcal{A})$ to matroid zwany matroidem grafowym.

Z definicji $\emptyset \in \mathcal{A}$.

Lemat 6.3. Niech $M = (S, \mathcal{A})$. Wszystkie maksymalne zbiory niezależne M mają tę samą liczbę.

Dowód. Niech $A, B \in \mathcal{A}$ będą maksymalnymi zbiorami niezależnymi M . Przypuśćmy, że $|B| > |A|$.

Z własności 4. istnieje $x \in B \setminus A$ taki, że $A \cup \{x\} \in \mathcal{A}$ – sprzeczność z maksymalnością A . $\square$

Problem $M = (S, \mathcal{A})$ – matroid, $w : S \rightarrow \mathbb{R}_+$ – funkcja wagi.

Dla $A \subseteq S$ niech $w(A) = \sum_{x \in A} w(x)$ – waga A .

Problem: Znaleźć zbiór niezależny o maksymalnej wadze w M .

$S = \{x_1, x_2, \dots, x_n\}$.

Algorytm 15: Algorytm Greedy

```

function GREEDY( $M, w$ )
1:   $n \leftarrow |S|$ 
2:   $A \leftarrow \emptyset$   $\triangleright A$  – rozwiązanie
3:  posortuj  $S$  w porządku nierosnącym względem wagi, tak żeby  $w(x_1) \geq w(x_2) \geq \dots \geq w(x_n)$ 
4:  for  $i \leftarrow 1$  to  $n$  do
5:    if  $A \cup \{x_i\} \in \mathcal{A}$  then
6:       $A \leftarrow A \cup \{x_i\}$ 
7:  return  $A$ 

```

Twierdzenie 6.4 (Rado, Edmonds). Algorytm Greedy znajduje zbiór niezależny o maksymalnej wadze.

Dowód. Niech $A = \{x_{i_1}, x_{i_2}, \dots, x_{i_k}\}$ będzie zbiorem zwróconym przez algorytm i $w(x_{i_1}) \geq w(x_{i_2}) \geq \dots \geq w(x_{i_k})$.

Najpierw pokażemy, że A jest maksymalnym zbiorem niezależnym.

Przypuśćmy, że istnieje $x_j \in S \setminus A$ taki, że $A \cup \{x_j\} \in \mathcal{A}$. Wtedy $j < i_k$, bo w przeciwnym wypadku algorytm dodałby x_j do A jako $(k+1)$ -szy element.

Niech l będzie najmniejszą liczbą taką, że $j < i_l$. Wtedy $w(x_{i_1}) \geq w(x_{i_2}) \geq \dots \geq w(x_{i_{l-1}}) \geq w(x_j) \geq w(x_{i_l})$.

$A \cup \{x_j\} \in \mathcal{A}$, więc z własności 3. $\{x_{i_1}, x_{i_2}, \dots, x_{i_{l-1}}, x_j\} \subseteq A \cup \{x_j\}$ jest zbiorem niezależnym.

Ale wtedy algorytm dodałby x_j zamiast x_{i_l} jako l -ty element dodany do A – sprzeczność.

Zatem A jest maksymalnym zbiorem niezależnym.

Rozważmy zbiór niezależny $B = \{y_1, y_2, \dots, y_m\} \in \mathcal{A}$ i załóżmy, że $w(y_1) \geq w(y_2) \geq \dots \geq w(y_m)$. Mamy $m \leq k$, bo A jest maksymalnym zbiorem niezależnym (Stw).

Pokażemy, że $w(B) \leq w(A)$.

Pokażemy, że dla każdego $j \leq m$ $w(y_j) \leq w(x_{i_j})$.

Przypuśćmy, że $w(y_j) > w(x_{i_j})$.

Rozważmy zbiory niezależne $A_{j-1} = \{x_{i_1}, \dots, x_{i_{j-1}}\} \subseteq A$, $B_j = \{y_1, \dots, y_j\} \subseteq B$.

$|B_j| = j > |A_{j-1}| = j - 1$, więc z własności 4. istnieje $y_s \in B_j \setminus A_{j-1}$ taki, że $A_{j-1} \cup \{y_s\} \in \mathcal{A}$.

Mamy $w(y_s) \geq w(y_j) > w(x_{i_j})$, więc istnieje $t \leq j$ takie, że $w(x_{i_1}) \geq w(x_{i_2}) \geq \dots \geq w(x_{i_{t-1}}) \geq w(y_s) > w(x_{i_t})$.

Z własności 3. $\{x_{i_1}, x_{i_2}, \dots, x_{i_{t-1}}, y_s\} \subseteq A_{j-1} \cup \{y_s\}$ jest zbiorem niezależnym.

Ale wtedy algorytm dodałby y_s zamiast x_{i_t} jako t -ty element dodany do A – sprzeczność.

$$\text{Zatem } \left. \begin{array}{l} w(y_1) \leq w(x_{i_1}) \\ w(y_2) \leq w(x_{i_2}) \\ \vdots \\ w(y_m) \leq w(x_{i_m}) \end{array} \right\} \Rightarrow w(B) = w(y_1) + \dots + w(y_m) \leq w(x_{i_1}) + \dots + w(x_{i_m}) \leq w(A). \quad \square$$

Złożoność czasowa algorytmu Greedy $n = |S|$

- linia 1: $\mathcal{O}(1)$
- linia 2: $\mathcal{O}(1)$
- linia 3: $\mathcal{O}(n \log n)$
- liczba iteracji pętli z linii 4 wynosi n
- linia 5: $f(n)$ – czas sprawdzenia warunku z linii 5
- linia 6: $\mathcal{O}(1)$
- linia 7: $\mathcal{O}(1)$

Problem szeregowania zadań

- $S = \{z_1, z_2, \dots, z_n\}$ – zbiór zadań o jednostkowym czasie wykonania
- $d_1, d_2, \dots, d_n$ – deadlines, $d_i \in \{1, \dots, n\}$ dla $i \in [n]$
- $w_1, w_2, \dots, w_n$ – kary, $w_i \geq 0$ dla $i \in [n]$

Płacimy karę w_i za zadanie z_i , jeżeli nie jest skończone w momencie d_i .

Problem: Znaleźć kolejność wykonania zadań minimalizującą sumę kar.

Założmy, że mamy pewien harmonogram. Zadanie z_i nazywamy terminowym w tym harmonogramie, jeśli jest zakończone do chwili d_i , w przeciwnym razie nazywamy je spóźnionym.

Nie płacimy kar za zadania terminowe, płacimy kary za zadania spóźnione.

Lemat 6.5. *Każdy harmonogram można przetransformować na harmonogram z taką samą sumą kar, w którym wszystkie zadania terminowe poprzedzają wszystkie zadania spóźnione oraz zadania terminowe są posortowane w kolejności niemalejącej względem deadline'ów.*

Dowód. 1. Niech z_i będzie zadaniem spóźnionym ukończonym w chwili $l_i > d_i$ i niech z_j będzie zadaniem terminowym ukończonym w chwili $l_j \leq d_j$ i niech z_i będzie przed z_j w harmonogramie, tzn. $l_i < l_j$.

Zamieniamy miejscami z_i i z_j . Po zamianie:

- z_i jest ukończone w chwili $l_j > l_i > d_i$, więc nadal jest spóźnione.
- z_j jest ukończone w chwili $l_i < l_j \leq d_j$, więc nadal jest terminowe.

Po pewnej liczbie takich zamian otrzymujemy harmonogram o tej samej sumie kar, w którym wszystkie zadania terminowe poprzedzają wszystkie zadania spóźnione.

2. Niech z_i, z_j będą zadaniami terminowymi takimi, że z_i jest ukończone w chwili $l_i \leq d_i$, z_j jest ukończone w chwili $l_j \leq d_j$, $l_i < l_j$, ale $d_i > d_j$.

Zamieniamy miejscami z_i i z_j . Po zamianie:

- z_i jest ukończone w chwili $l_j \leq d_j < d_i$, więc z_i jest nadal terminowe.
- z_j jest ukończone w chwili $l_i < l_j \leq d_j$, więc z_j jest nadal terminowe.

Po pewnej liczbie takich zamian otrzymujemy harmonogram spełniający warunki lematu.

□